

I want to set python development environment in this machin using VS code. Please help me how to set it up. I have already VS code and python installed on this machin.

To set up a Python development environment in Visual Studio Code on your machine, follow these steps.

Step 1: Verify Python Installation

Open Command Prompt (cmd) and run:

```
python --version
```

or

```
py --version
```

You should see something like:

```
Python 3.12.2
```

Also check pip:

```
pip --version
```

If these commands work, Python is installed correctly.

Step 2: Open VS Code

Start Visual Studio Code.

Step 3: Install Python Extension in VS Code

Inside VS Code:

1. Click Extensions icon on left sidebar
2. Search:

Python

3. Install extension from:

Microsoft

You should install:

- Python
- Pylance (usually auto-installed)

These provide:

- IntelliSense
 - Auto-completion
 - Debugging
 - Error checking
-

Step 4: Create a Project Folder

Example:

D:\PythonProjects\MyFirstApp

Open this folder in VS Code:

File → Open Folder

Step 5: Create a Python File

Create:

app.py

Add:

```
print("Hello Python")
```

Step 6: Select Python Interpreter

This is VERY important.

Press:

```
Ctrl + Shift + P
```

Search:

```
Python: Select Interpreter
```

Choose your installed Python version.

Example:

```
Python 3.12
```

Step 7: Run Python Program

Method 1:

Click top-right  Run button.

Method 2:

Open terminal in VS Code:

```
Terminal → New Terminal
```

Run:

```
python app.py
```

Output:

```
Hello Python
```

Step 8: Create Virtual Environment (Recommended)

Inside project terminal:

```
python -m venv venv
```

This creates:

```
venv/
```

Step 9: Activate Virtual Environment

On Windows:

```
venv\Scripts\activate
```

You should see:

```
(venv)
```

Now all packages install locally to the project.

Step 10: Install Packages

Example:

```
pip install numpy pandas flask
```

Step 11: Freeze Dependencies

Create requirements file:

```
pip freeze > requirements.txt
```

Step 12: Enable Auto Formatting (Optional but Recommended)

Install formatter:

```
pip install black
```

Then in VS Code:

1. Press `Ctrl + Shift + P`
2. Search:

```
Format Document With
```

3. Choose:

```
Black Formatter
```

Step 13: Useful VS Code Settings

Open Settings:

```
Ctrl + ,
```

Search and enable:

- Format On Save
 - Python Linting
 - Auto Save (optional)
-

Recommended Folder Structure

```
MyFirstApp/  
|  
├─ venv/  
├─ app.py  
├─ requirements.txt  
├─ data/  
├─ src/  
└─ tests/
```

Useful Python Packages for You

Since you are learning AI and backend development, install:

```
pip install numpy pandas matplotlib scikit-learn jupyter
```

For API development:

```
pip install fastapi uvicorn
```

For AI:

```
pip install transformers torch
```

Verify Everything Works

Create:

```
import numpy as np

print(np.array([1, 2, 3]))
```

Run:

```
python app.py
```

Bonus: Enable Jupyter Notebook in VS Code

Install extension:

Jupyter

Then create:

test.ipynb

Useful for:

- AI
- Machine Learning
- Data Science
- PySpark learning

Recommended Next Steps for You

Based on your background in C#, .NET, AI, and backend development:

1. Learn Python basics
2. Learn virtual environments deeply
3. Learn package management
4. Learn Jupyter
5. Learn FastAPI
6. Learn NumPy + Pandas

7. Learn PyTorch + Transformers

Official websites:

- [Python Official Website](#)
- [VS Code Official Website](#)
- [Python VS Code Extension](#)

Your setup is already good for starting Machine Learning development.

You currently have:

- [Python 3.11.9](#)
- [pip 26.1.1](#)

That is a solid modern setup.

For ML and AI practice using advanced libraries, you should now do these additional setup steps.

1. Create a Dedicated ML Workspace

Create a folder like:

```
D:\AIProjects
```

Inside it:

```
D:\AIProjects\MLPractice
```

Open this folder in Visual Studio Code.

2. Create Virtual Environment (VERY Important)

Inside VS Code terminal:

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

You should see:

```
(venv)
```

This isolates ML libraries from system Python.

3. Upgrade pip Inside venv

Run:

```
python -m pip install --upgrade pip
```

4. Install Core ML Libraries

Start with:

```
pip install numpy pandas matplotlib scikit-learn jupyter notebook
```

Then install deep learning libraries:

```
pip install torch torchvision torchaudio
```

Then NLP/LLM libraries:

```
pip install transformers datasets accelerate sentencepiece
```

5. Install Useful Developer Tools

```
pip install ipykernel black pylint
```

These help with:

- Notebook execution
- Formatting
- Code quality

6. Install Jupyter Extension in VS Code

Install these VS Code extensions:

- Python
- Pylance
- Jupyter

Then create:

practice.ipynb

Notebook support is extremely useful for ML experimentation.

7. Verify PyTorch Installation

Run:

```
import torch

print(torch.__version__)
print(torch.cuda.is_available())
```

If output is:

False

It means GPU is not enabled yet (normal for now).

8. Check Whether You Have NVIDIA GPU

Open Command Prompt:

```
nvidia-smi
```

If command works, you have NVIDIA GPU support.

If not, you may still use CPU training.

9. If You Have NVIDIA GPU → Install CUDA Version of PyTorch

Official install page:

PyTorch Install Guide

This gives MUCH faster ML training.

10. Recommended Libraries for Your Learning Path

Since you want advanced ML/AI:

Classical ML

```
pip install scikit-learn xgboost lightgbm
```

Deep Learning

```
pip install tensorflow keras
```

OR use PyTorch (recommended nowadays).

NLP / LLM

```
pip install transformers sentence-transformers
```

Data Visualization

```
pip install seaborn plotly
```

11. Recommended Folder Structure

```
MLPractice/  
|  
├─ venv/  
├─ notebooks/  
├─ datasets/  
├─ models/  
├─ src/  
├─ experiments/  
└─ requirements.txt
```

12. Save Dependencies

After installations:

```
pip freeze > requirements.txt
```

13. Important VS Code Settings

Enable:

- Python > Terminal: Activate Environment
- Format On Save
- Jupyter Notebook support

14. Best Starting Practice Flow

You can now practice:

1. NumPy
2. Pandas
3. Matplotlib
4. Scikit-learn
5. PyTorch
6. Transformers/BERT
7. Fine-tuning models

8. RAG applications
 9. AI APIs
-

15. One Important Recommendation

For serious ML work later, consider:

- 16GB+ RAM
- NVIDIA GPU
- SSD storage

Because models like PyTorch and TensorFlow can consume high memory.

16. Your Next Immediate Commands

Run these now:

```
mkdir D:\AIProjects
cd D:\AIProjects

mkdir MLPractice
cd MLPractice

python -m venv venv

venv\Scripts\activate
```

Then:

```
pip install numpy pandas matplotlib scikit-learn jupyter
```

After this, your ML environment is ready for learning and experimentation.

```

PS C:\projects\MLPractice> nvidia-smi
Fri May 15 21:05:27 2026
+-----+
+-----+
| NVIDIA-SMI 591.86                Driver Version: 591.86
| CUDA Version: 13.1               |
+-----+-----+
+-----+
| GPU  Name           Driver-Model | Bus-Id      Disp.A
| Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-

```

Excellent — your machine is already capable of GPU-based Machine Learning training.

You have:

- NVIDIA Quadro T1000 GPU
- CUDA 13.1 support
- 4GB VRAM

This is good for:

- Learning ML/DL
- Small to medium neural networks
- BERT inference
- Fine-tuning small transformer models
- Computer vision practice
- PyTorch experiments
- Local AI development

For very large LLM training (like 7B+ models), 4GB VRAM will be limited, but for learning and most practice projects, this setup is absolutely usable.

What You Should Do Next

Now install GPU-enabled PyTorch.

Step 1: Activate Your venv

Inside project folder:

```
venv\Scripts\activate
```

Step 2: Install GPU PyTorch

Go to:

[PyTorch Install Page](#)

For your setup, install using pip.

Most stable option currently:

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu128
```

Even though your driver supports CUDA 13.1, PyTorch officially provides builds for CUDA 12.x currently. That is normal.

Step 3: Verify GPU Detection

Create:

```
import torch

print("PyTorch version:", torch.__version__)
print("CUDA Available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU Name:", torch.cuda.get_device_name(0))
```

Run:

```
python test.py
```

Expected:

```
CUDA Available: True  
GPU Name: Quadro T1000
```

Step 4: Install Core AI Libraries

Now install:

```
pip install transformers datasets accelerate sentence-transformers
```

Also:

```
pip install jupyter notebook matplotlib pandas scikit-learn
```

Step 5: Install VS Code Extensions

Inside Visual Studio Code install:

- Python
- Pylance
- Jupyter

Step 6: Recommended First AI Notebook

Create:

```
notebooks/bert_test.ipynb
```

Test this:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("I love learning AI")

print(result)
```

This verifies:

- Transformers
- Model downloading
- GPU inference
- HuggingFace setup

Important Note About Your GPU

Your GPU VRAM:

4GB

So avoid initially:

- Llama 7B local training
- Large diffusion models
- Huge batch sizes

Instead start with:

- BERT
- DistilBERT
- TinyLlama
- Phi models
- Classical ML
- CNN practice

Recommended Settings for Your GPU

For training:

```
batch_size = 4 or 8
```

Use smaller models initially.

Very Useful Additional Tools

Install:

```
pip install tensorboard
```

For training visualization.

Recommended Learning Sequence For You

Since you already know programming and backend development:

Phase 1

- NumPy
 - Pandas
 - Matplotlib
 - Scikit-learn
-

Phase 2

- Neural Networks
 - PyTorch tensors
 - Training loops
 - CNN
 - RNN
-

Phase 3

- Transformers
 - BERT
 - Attention
 - HuggingFace
-

Phase 4

- RAG
 - Vector DB
 - LangChain
 - AI APIs
-

One More Recommendation

Install Git if not already installed.

Check:

```
git --version
```

If not installed:

[Git Official Website](#)

Git is extremely important for AI projects.

Your Setup Status

Current status:

Component	Status
Python	 Ready
pip	 Ready

Component	Status
VS Code	✓ Ready
NVIDIA GPU	✓ Ready
CUDA Driver	✓ Ready
ML Development	✓ Ready
Deep Learning	✓ Ready

You are now ready to start serious AI/ML practice locally.

```
still i can see (venv) PS C:\projects\MLPractice> python
test.py
PyTorch Version: 2.12.0+cpu
CUDA Available: False
```

You are inside the virtual environment correctly because you can see:

```
(venv)
```

The issue is different:

You installed the CPU version of PyTorch instead of the CUDA/GPU version.

That is why:

```
PyTorch Version: 2.12.0+cpu
CUDA Available: False
```

appears.

Fix: Install CUDA-enabled PyTorch

Step 1: Uninstall Current CPU Version

Inside your activated venv:

```
pip uninstall torch torchvision torchaudio
```

Run until all are removed.

Step 2: Install CUDA Version

Now install GPU-enabled build:

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu128
```

Wait for download and installation.

This package contains:

- CUDA runtime
 - GPU support
 - Optimized kernels
-

Step 3: Verify Again

Run:

```
import torch

print("PyTorch version:", torch.__version__)
print("CUDA Available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

Expected output:

```
CUDA Available: True
GPU: Quadro T1000
```

Step 4: Verify Tensor on GPU

Test this:

```
import torch

x = torch.rand(3, 3).cuda()

print(x)
```

If it works without error, GPU acceleration is working.

Why This Happened

By default:

```
pip install torch
```

usually installs CPU-only build.

GPU support requires explicitly installing CUDA build.

Important Note About CUDA Versions

You have:

```
CUDA Driver: 13.1
```

But PyTorch ships its own CUDA runtime internally.

So installing:

```
cu128
```

is correct and compatible.

You do NOT need full CUDA Toolkit separately right now.

Optional (Later)

You may later install:

- `CUDA Toolkit`
- `cuDNN`

But for most PyTorch learning, not required initially.

After GPU Works

Install these:

```
pip install transformers datasets accelerate sentence-transformers
```

Then you can start:

- BERT
- Fine-tuning
- Embeddings
- NLP
- CNNs
- AI apps

with GPU acceleration.